
● SAVOL 21:

SEABORN KUTUBXONASINING MATPLOTLIB DAN USTUNLIKLARI: STATISTIK VIZUALIZATSIYALAR, HEATMAP, PAIRPLOT, BOXPLOT. NIMA UCHUN MA'LUMOTLARNI TUSHUNISHDA VIZUALIZATSIYA MUHIM?

1. Vizualizatsiya Nima Uchun Muhim? — Falsafiy Asos

Inson miyasi raqamlarni emas, **shakllarni, ranglarni, naqshlarni** tez idrok etadi. Bu evolyutsion haqiqat:

Raqamlar jadvali:

23, 45, 12, 67, 34, 89, 23, 45, 11, 78, 90, 34...
→ Inson bu raqamlarda naqsh ko'ra olmaydi

Grafik:

→ Bir qarashda tendensiya, anomaliya, taqsimot ko'rinadi

Anscombe Kvarteti — Eng Mashhur Misol:

1973 yilda statistik Anscombe to'rtta turli xil dataset yaratdi. Ularning barchasi **bir xil statistik ko'rsatkichlarga** ega:

To'rt dataset uchun:

O'rtacha X: 9.0 (barchasi bir xil)
O'rtacha Y: 7.5 (barchasi bir xil)
X dispersiyasi: 11.0 (barchasi bir xil)
Y dispersiyasi: 4.12 (barchasi bir xil)
Korrelyatsiya: 0.816 (barchasi bir xil)

Raqamlar bir xil → Grafiklar mutlaqo boshqacha!

Dataset 1 → To'g'ri chiziq korrelyatsiyasi
Dataset 2 → Egri chiziq (kvadratik)
Dataset 3 → Aniq chiziq + bitta anomaliya
Dataset 4 → Vertikal chiziq + bitta chetdagi nuqta

Xulosa: Raqamlar yolg'on gapiradi, grafik haqiqatni ko'rsatadi.

2. Seaborn nima va Matplotlib bilan munosabati

Seaborn — Matplotlib ustiga qurilgan yuqori darajali statistik vizualizatsiya kutubxonasi. Matplotlib **mexanizm** bo'lsa, Seaborn — **statistik til**.

Matplotlib: Qanday chizishni biladi (chiziqlar, ranglar, o'lchamlar)
Seaborn: Nima chizishni biladi (taqsimot, korrelyatsiya, tendensiya)

Munosabat:

Seaborn → Matplotlib ustida ishlaydi

Seaborn grafiklari → aslida Matplotlib Figure/Axes
Seaborn dan keyin → plt.title(), ax.set_xlabel() ishlaydi

3. Seaborn ning Matplotlib dan Ustunliklari

Ustunlik 1 — Kamroq kod, ko'proq ma'lumot:

```
# Matplotlib da boxplot:  
fig, ax = plt.subplots()  
data_by_category = [df[df["kun"]==k]["qiymat"].values for k in df["kun"].unique()]  
ax.boxplot(data_by_category)  
ax.set_xticklabels(df["kun"].unique())  
# + Rang, stil, statistika qo'shish uchun yana 10+ qator  
  
# Seaborn da – bir qator:  
sns.boxplot(data=df, x="kun", y="qiymat", hue="guruh")  
# Rang, statistika, Legend – avtomatik!
```

Ustunlik 2 — Pandas DataFrame bilan to'g'ridan-to'g'ri ishlash:

Matplotlib: numpy array talab qiladi → DataFrame dan qo'lda ajratish
Seaborn: DataFrame ni to'g'ridan-to'g'ri qabul qiladi

```
sns.scatterplot(data=df, x="yosh", y="daromad", hue="shahar",  
size="tajriba")  
↑ Bu bir qatorda 4 ta o'zgaruvchini vizualizatsiya qiladi!
```

Ustunlik 3 — Statistik hisob-kitob avtomatik:

Seaborn grafiklar ichida quyidagi statistikalarni o'zi hisoblaydi:

- O'rtacha qiymat va ishonch intervali (confidence interval)
- Mediana, kvartillar (boxplot)
- Yadro zichligi (KDE – Kernel Density Estimation)
- Regressiya chizig'i (regplot)

Matplotlib da bularni qo'lda hisoblash kerak!

Ustunlik 4 — Chiroyli standart ko'rinish:

Matplotlib standarti → oddiy, bezaksiz
Seaborn standarti → professional, ilmiy jurnal sifatida

```
sns.set_theme() → bir qatorda professional ko'rinish  
sns.set_palette("husl") → moslashtirilgan rang palatrasi
```

4. Heatmap — Issiqlik Xaritasi

Nazariy asos: Heatmap — ikki o'lchovli matritsa ma'lumotlarini rang intensivligi orqali ko'rsatadi. Inson ko'zi rang farqini raqam farqidan 10x tezroq idrok etadi.

Qachon ishlatiladi:

Korrelyatsiya matritsasi	→ qaysi o'zgaruvchilar bog'liq?
Vaqt × Kategoriya	→ qaysi kunda qaysi tovar ko'p sotilgan?
Geografik ma'lumot	→ qaysi hududda ko'rsatkich yuqori?
Confusion matrix (ML)	→ model qaysi klassni adashtirmoqda?

Korrelyatsiya Heatmap — tushuntirish:

Korrelyatsiya koeffitsienti:

- +1.0 → Mukammal musbat bog'liqlik
- +0.7 → Kuchli musbat bog'liqlik
- +0.3 → Zaif musbat bog'liqlik
- 0.0 → Bog'liqlik yo'q
- 0.3 → Zaif manfiy bog'liqlik
- 0.7 → Kuchli manfiy bog'liqlik
- 1.0 → Mukammal manfiy bog'liqlik

Heatmap da:

- To'q qizil → +1.0 ga yaqin
- Oq → 0.0 ga yaqin
- To'q ko'k → -1.0 ga yaqin

```
# Korrelyatsiya heatmap:  
korrelatsiya = df.corr()
```

```
sns.heatmap(  
    korrelatsiya,  
    annot=True,           # har katakda raqam ko'rsatish  
    fmt=".2f",            # 2 xona aniqlik  
    cmap="coolwarm",      # rang xaritasi: qizil-oq-ko'k  
    vmin=-1, vmax=1,      # rang chegaralari  
    center=0,             # oq rang 0 da  
    square=True,          # kvadrat kataklar  
    linewidths=0.5,      # kataklararo chiziq  
)
```

cmap tanlash falsafasi:

- "coolwarm" → Diverging (ikki qarama-qarshi tomon) – korrelyatsiya uchun
- "viridis" → Sequential (bir yo'nalish) – davomiy qiymatlar uchun
- "Blues" → Sequential, bir rang – oddiy taqsimot uchun
- "Set2" → Categorical (kategoriyalar) – guruhlar uchun

5. Pairplot — Ko'p O'zgaruvchili Tahlil

Nazariy asos: n ta o'zgaruvchi uchun n×n grafik matritsasi yaratadi. Diagonal → har o'zgaruvchining taqsimoti, diagonal emas → ikki o'zgaruvchi orasidagi munosabat.

Qachon ishlatiladi:

Ma'lumotlarni dastlabki o'rganish (EDA – Exploratory Data Analysis)
Qaysi o'zgaruvchilar korrelyatsiyaga ega – vizual aniqlash

Guruhlar orasidagi farqni ko'rish (hue parametri bilan)
Machine Learning uchun feature tanlash

Pairplot nima ko'rsatadi:

Diagonal (scatter emas):

- Har o'zgaruvchining yolg'iz taqsimoti
- KDE yoki histogram

Diagonal emas (scatter):

- Chap pastga → o'ng yuqoriga: musbat korrelyatsiya
- Aniq klasterlar: ma'lumotda guruhlar bor
- Kuchli naqsh yo'q: o'zgaruvchilar mustaqil

```
sns.pairplot(  
    df,  
    hue="tur",          # Ranglar bilan guruhlar  
    diag_kind="kde",     # Diagonal: "hist" yoki "kde"  
    plot_kws={"alpha": 0.6}, # Scatter nuqtalar shaffof  
    diag_kws={"fill": True},  # KDE to'ldirilgan  
    corner=True          # Faqat pastki uchburchak (tezroq)  
)
```

Nazariy tushuncha — KDE (Kernel Density Estimation):

Histogram → Ma'lumotni "chelaklar"ga bo'ladi → diskret, qo'pol
KDE → Ma'lumotni silliq egri chiziq bilan ifodalaydi → uzluksiz

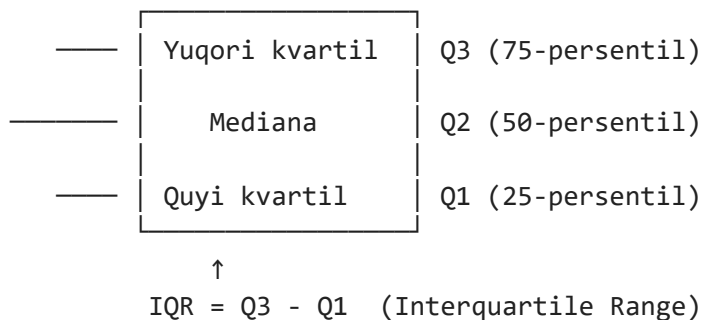
KDE aslida: har nuqta atrofida kichik "qo'ng'iroq" shaklida taqsimot
qo'yiladi, barchasi qo'shiladi → silliq taqsimot egri chizig'i

Qachon KDE: taqsimot shaklini aniq ko'rish kerak
Qachon hist: aniq chastotalarni bilish kerak

6. Boxplot — Statistik Taqqoslash

Nazariy asos: Boxplot (quti grafik) — beshta statistik ko'rsatkichni bir vaqtda ko'rsatadi. John Tukey (1969) tomonidan ixtiro qilingan.

Boxplot anatomiyasi:



Whisker (mo'ylov):

Yuqori: $Q3 + 1.5 \times IQR$
Quyi: $Q1 - 1.5 \times IQR$

Whiskerdan tashqari nuqtalar → ANOMALIYALAR (outliers)

Boxplot nimani ko'rsatadi:

Quti balandligi → Ma'lumot tarqoqligi (keng = ko'p tarqoq)
Mediana joylashuvi → Nosimmetriklik (chapga/o'ngga og'ish)
Whisker uzunligi → Dumlar og'irligi
Outlier nuqtalar → Anomal qiymatlar

Violin plot — Boxplot ning kuchli varianti:

Boxplot: Faqat 5 statistik ko'rsatkich
Violin plot: Boxplot + taqsimot shakli (KDE)

Violin plot qachon afzal:

- Taqsimot bir cho'qqili yoki ko'p cho'qqilimi — ko'rish kerak bo'lsa
- Guruhlar ichki tuzilishini solishtirish kerak bo'lsa

Boxplot:

```
sns.boxplot(  
    data=df,  
    x="kategoriya",  
    y="qiymat",  
    hue="guruh",      # Rang bo'yicha ajratish  
    palette="Set2",    # Rang palatrasi  
    width=0.6,        # Quti kengligi  
    flierprops={"marker": "o", "markersize": 4} # Outlier ko'rinishi  
)
```

Violin plot:

```
sns.violinplot(  
    data=df,  
    x="kategoriya",  
    y="qiymat",  
    inner="box",      # Ichida: "box", "quart", "point"  
    split=True,       # hue bo'lsa: ikki tomonga bo'lish  
)
```

7. Seaborn ning Boshqa Muhim Grafiklari

Regplot — Regressiya chizig'i bilan Scatter:

Nazariy asos: Scatter plot + regressiya chizig'i +
ishonch intervali (shaded area).
Ikki o'zgaruvchi orasidagi chiziqli munosabatni ko'rsatadi.

ishonch intervali nima:

- 95% CI → "Haqiqiy regressiya chizig'i 95% ehtimollik bilan shu soyali zonada yotadi"
- Keng zona → ma'lumot kam yoki tarqoq
- Tor zona → ma'lumot ko'p va aniq

Catplot — Kategoriyali Ma'lumotlar:

Kategoriyali X + raqamli Y → catplot

kind="strip" → Har nuqta alohida (jitter bilan)
kind="swarm" → Nuqtalar bir-biriga tegmasligi uchun tarqaladi
kind="box" → Boxplot
kind="violin" → Violin plot
kind="bar" → O'rtacha + CI

Qachon qaysi:

Kam ma'lumot → "strip" yoki "swarm" (har nuqta ko'rinadi)
Ko'p ma'lumot → "box" yoki "violin" (umumiy tuzilish ko'rinadi)

Displot — Taqsimot Tahlili:

kind="hist" → Histogram
kind="kde" → KDE egri chizig'i
kind="ecdf" → Kumulyativ taqsimot funksiyasi

ECDF nima:

"X qiymatdan kichik ma'lumotlarning ulushi"

Masalan: $ECDF(70) = 0.6$ → ma'lumotlarning 60% 70 dan kichik

Gistogrammadan afzalligi: bins tanlanmaydi, barcha ma'lumot ko'rinadi

8. Seaborn Stillar va Kontekst

Stil (grafik ko'rinishi):

```
sns.set_style("whitegrid") # Oq fon + setka ← ilmiy uchun
sns.set_style("darkgrid") # To'q fon + setka ← taqdimot uchun
sns.set_style("white")    # Toza oq fon
sns.set_style("dark")     # To'q fon
sns.set_style("ticks")    # Faqat o'q chiziqlari
```

Kontekst (o'lcham – qayerda ko'rsatiladi):

```
sns.set_context("paper") # Ilmiy maqola – kichik
sns.set_context("notebook") # Jupyter – o'rta (standart)
sns.set_context("talk") # Taqdimot – kattaroq shrift
sns.set_context("poster") # Poster – eng katta
```

Nazariy asos — nima uchun kontekst muhim:

Bir xil grafik turli joyda turlicha ko'rinishi kerak:

Ilmiy jurnal → kichik, aniq, zich

Taqdimot slayd → katta shrift, qalin chiziqlar, uzoqdan ko'rinsin

Poster → juda katta, minimal matn

Kontekst faqat o'lchamlarni o'zgartiradi – grafik mazmuni bir xil

9. Qaysi Grafik Qaysi Savol Uchun?

"Bu ma'lumot qanday taqsimlangan?" → histogram / KDE / displot
"Ikkisi orasida bog'liqlik bormi?" → scatter / regplot
"Ko'p o'zgaruvchi o'rtasida nima?" → pairplot / heatmap
"Guruhlar orasida farq bormi?" → boxplot / violinplot
"Vaqt o'tishi bilan o'zgarish?" → lineplot
"Kategoriyalar ulushi?" → barplot / pie
"Anomaliyalar bormi?" → boxplot / scatter
"Ko'p o'zgaruvchi korrelyatsiyasi?" → heatmap

10. Xulosa

Vizualizatsiya nima uchun muhim:

- Inson miyasi grafik naqshlarni raqamlardan tez ko'radi
- Anscombe kvarteti: bir xil statistika, boshqacha grafik
- Anomaliya, tendensiya, taqsimot – ko'z bilan ko'rinadi

Seaborn vs Matplotlib:

- Matplotlib → past daraja, to'liq nazorat, ko'p kod
- Seaborn → yuqori daraja, statistik avtomatika, kam kod
- Seaborn Matplotlib ustida ishlaydi – ular raqib emas

Asosiy grafik turlari:

- heatmap → Matritsa, korrelyatsiya – rang intensivligi
- pairplot → Ko'p o'zgaruvchi – $n \times n$ grafik matritsasi
- boxplot → Statistik taqqoslash – 5 ko'rsatkich bir vaqtda
- violin → Boxplot + taqsimot shakli

Asosiy qoida:

- ✓ Grafik savol uchun tanlanadi – savol yo'q = grafik yo'q
- ✓ Kamroq grafik, ko'proq ma'no – bezak emas, tushunish
- ✓ Har grafik sarlavha + o'q nomlari + legend bo'lishi shart

💡 **Eslab qol:** Vizualizatsiya — bu bezak emas, **tushunish vositasi**. Eng yaxshi grafik — ko'rgan odam "Voy, qiziq!" deb qolmaydigan, balki "Endi tushundim!" deb aytadigan grafik!

● SAVOL 22:

CHIZIQLI REGRESSIYA (LINEAR REGRESSION) ALGORITMI: MATEMATIK ASOS (OLS), MODEL O'RGATISH VA BASHORAT QILISH. KOEFFITSIYENTLARNI TALQIN ETISH VA MODELNI BAHOLASH (R^2 , MSE).

1. Chiziqli Regressiya — Nazariy Asos

Chiziqli regressiya — bir yoki bir nechta **mustaqil o'zgaruvchilar** (X) asosida **bog'liq o'zgaruvchini** (Y) bashorat qilish usuli. Asosiy g'oya: ma'lumotlar orasidan **eng yaxshi to'g'ri chiziq** topish.

Hayotiy misollar:

$X = \text{ta'lim yillari} \rightarrow Y = \text{oylik maosh}$

$X = \text{uy maydoni (m}^2\text{)} \rightarrow Y = \text{uy narxi}$

$X = \text{reklama xarajati} \rightarrow Y = \text{sotuvlar hajmi}$

$X = \text{temperatura} \rightarrow Y = \text{elektr sarfi}$

Matematik shakl:

Oddiy chiziqli regressiya (1 ta X):

$$Y = \beta_0 + \beta_1 \cdot X + \varepsilon$$

Ko'p o'zgaruvchili (n ta X):

$$Y = \beta_0 + \beta_1 \cdot X_1 + \beta_2 \cdot X_2 + \dots + \beta_n \cdot X_n + \varepsilon$$

Bu yerda:

$Y \rightarrow$ Bashorat qilinuvchi qiymat (target)

$X \rightarrow$ Kiruvchi o'zgaruvchi (feature)

$\beta_0 \rightarrow$ Intercept (kesma – $X=0$ da Y qiymati)

$\beta_1 \rightarrow$ Slope (qiyalik – X 1 ga oshsa Y qancha o'zgaradi)

$\varepsilon \rightarrow$ Xato (model tushuntirolmagan qism)

2. OLS — Eng Kichik Kvadratlar Usuli

OLS (Ordinary Least Squares) — chiziqli regressiyaning matematik yechimi. Maqsad: **haqiqiy qiymatlar va bashorat qiymatlar orasidagi farqlar kvadratlari yig'indisini minimallashtirish.**

Har nuqta uchun xato (qoldiq – residual):

$$e_i = Y_i - \hat{Y}_i$$

$Y_i \rightarrow$ haqiqiy qiymat

$\hat{Y}_i \rightarrow$ model bashorati

OLS minimallashtirish kerak bo'lgan:

$$SSE = \sum (Y_i - \hat{Y}_i)^2 \rightarrow \text{minimal bo'lsin}$$

Nima uchun kvadrat?

$\rightarrow$ Musbat va manfiy xatolar bir-birini bekor qilmasin

$\rightarrow$ Katta xatolarga ko'proq jarima (kvadrat katta farqni kuchaytiradi)

$\rightarrow$ Matematik differentsiallash uchun qulay

Nima uchun absolyut qiymat emas?

$|e_i| \rightarrow$ MAE – differentsiallashda qiyin (0 da uzilish)

$e_i^2 \rightarrow$ MSE – har yerda differentsiallanadi $\rightarrow$ yechish oson

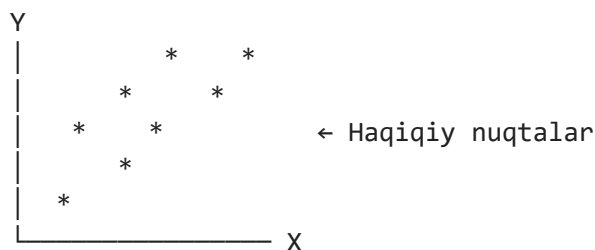
OLS analitik yechimi (matritsa ko'rinishida):

$$\beta = (X^T X)^{-1} \cdot X^T \cdot Y$$

Bu formula β_0 va β_1 ni bir qadamda topadi!

Iteratsiya shart emas – bu OLS ning kuchli tomoni.

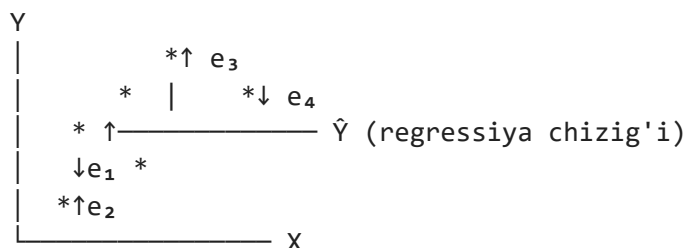
3. Geometrik Talqin



Chiziqli regressiya:

- Bu nuqtalar orasidan "eng yaxshi" to'g'ri chiziq topadi
- "Eng yaxshi" = vertikal masofalar kvadrati minimalli

Vertikal masofa = residual (qoldiq)



4. Koeffitsiyentlarni Talqin Etish

Bu qism **eng muhim nazariy qism** — koeffitsiyentlarni noto'g'ri talqin etish keng tarqalgan xato.

β_0 — Intercept (Kesma):

Talqin: "X = 0 bo'lganda Y ning kutilgan qiymati"

Misol: $Y = 20,000 + 500 \cdot X$

(Y = maosh ming so'mda, X = tajriba yillarda)

$\beta_0 = 20,000 \rightarrow$ tajribasiz xodim (X=0) ning maoshi 20,000

Ehtiyot:

Ko'p holatlarda X=0 mantiqsiz (uy maydoni 0 m² bo'lmaydi)

Bunday holda β_0 ni talqin qilish xato — u faqat

chiziqni "joylashtirish" uchun kerak

β_1 — Slope (Qiyalik):

Talqin: "X 1 birlikka oshsa, Y o'rtacha β_1 ga o'zgaradi, boshqa barcha omillar o'zgarmas bo'lganda"

Misol: $Y = 20,000 + 500 \cdot X$

$\beta_1 = 500 \rightarrow$ Har yil tajriba maoshni o'rtacha 500 ga oshiradi

Ko'p o'zgaruvchili holatda:

$Y = 20,000 + 500 \cdot X_1 + 2,000 \cdot X_2$

(X_1 = tajriba, X_2 = ta'lim darajasi)

$\beta_1 = 500 \rightarrow X_2$ o'zgarmas bo'lganda, tajriba 1 yilga oshsa maosh 500 ga oshadi

"Boshqa omillar o'zgarmas" – bu juda muhim shart!

Standartlashtirilgan ko'effitsiyentlar:

Muammo: X_1 = tajriba (0-40 yil), X_2 = ta'lim (0-5 daraja)

$\beta_1 = 500$, $\beta_2 = 2,000$

"Ta'lim muhimroq" deb bo'lmaydi – birliklar har xil!

Yechim – Standardizatsiya (Z-score):

$X_i^* = (X_i - \mu) / \sigma$

Standartlashtirilgan $\beta \rightarrow$ "Qaysi o'zgaruvchi muhimroq?" – to'g'ri taqqoslash

5. Model Baholash — R^2

R^2 (Determinatsiya ko'effitsienti) — model ma'lumotdagi o'zgarishning qancha ulushini tushuntiradi.

Matematik ta'rif:

$SS_{tot} = \sum (Y_i - \bar{Y})^2$ ← Umumiy o'zgarish (Y o'rtachadan farqi)

$SS_{res} = \sum (Y_i - \hat{Y}_i)^2$ ← Tushuntirilmagan o'zgarish (qoldiqlar)

$R^2 = 1 - SS_{res} / SS_{tot}$

Talqin:

$R^2 = 0.85 \rightarrow$ Model Y dagi o'zgarishlarning 85% ini tushuntiradi

$R^2 = 1.0 \rightarrow$ Mukammal moslik (amalda shubhali – overfit!)

$R^2 = 0.0 \rightarrow$ Model hech narsani tushuntirmaydi

$R^2 < 0 \rightarrow$ Model o'rtachadan ham yomon (juda yomon!)

R^2 qiymatini talqin etish:

Soha	Yaxshi R^2
Ijtimoiy fanlar	0.3 – 0.5 (insonlar bashorat qilib bo'lmaydi)
Iqtisodiyot	0.5 – 0.7
Muhandislik	0.8 – 0.95
Fizika	0.99+ (qonunlar aniq)

$R^2 = 0.9$ ijtimoiy fanda ajoyib,

$R^2 = 0.9$ fizikada – yomon model!

R^2 ning cheklovi — Adjusted R^2 :

Muammo: Har yangi o'zgaruvchi qo'shilsa R^2 oshadi
(hatto mantiqsiz o'zgaruvchi ham!)

Yechim – Adjusted R^2 :

$R^2_{adj} = 1 - (1 - R^2) \cdot (n - 1) / (n - k - 1)$

$n \rightarrow$ kuzatuvlar soni

$k \rightarrow$ o'zgaruvchilar soni

Yangi o'zgaruvchi R^2 ni oshirsa lekin
Adjusted R^2 ni tushirsa → o'zgaruvchi keraksiz!

6. Model Baholash — MSE, RMSE, MAE

MSE (Mean Squared Error – O'rtacha Kvadratik Xato):

$$MSE = (1/n) \cdot \sum (Y_i - \hat{Y}_i)^2$$

- Katta xatolarga ko'proq jarima (kvadrat tufayli)
- Birlik: Y^2 (agar Y = so'm bo'lsa, MSE = so'm²)
- Shuning uchun interpretatsiya qiyin

RMSE (Root MSE – Kvadrat ildiz):

$$RMSE = \sqrt{MSE}$$

- MSE ning kvadrat ildizi → Y bilan bir xil birlik
- "O'rtacha xato so'mda qancha?" → RMSE javob beradi
- Katta xatolarga sezgir (outlierlar ta'sir qiladi)

MAE (Mean Absolute Error):

$$MAE = (1/n) \cdot \sum |Y_i - \hat{Y}_i|$$

- O'rtacha mutlaq xato
- Outlierga kam sezgir (kvadrat yo'q)
- Talqin qilish oson: "O'rtacha X so'm adashaman"

Qachon qaysi:

Outlier muhim (yo'qolgan → katta zarar) → RMSE

Outlier muhim emas, median xato → MAE

Nisbiy taqqoslash → R^2

7. Chiziqli Regressiya Taxminlari (Assumptions)

OLS to'g'ri ishlashi uchun bir nechta shart bajarilishi kerak. Bularni bilmaslik — noto'g'ri model va noto'g'ri talqin olib keladi:

1. Chiziqlilik:

X va Y orasidagi munosabat chiziqli bo'lishi kerak

Tekshirish → scatter plot (egri bo'lsa → transformatsiya kerak)

2. Mustaqillik:

Kuzatuvlar bir-biridan mustaqil bo'lishi kerak

Buzilishi → vaqt qatori ma'lumotlari (autocorrelation)

3. Gomoskedastiklik (bir xil dispersiya):

Qoldiqlarning tarqoqligi X bo'yicha bir xil bo'lishi kerak

Buzilishi → "geteroskodastiklik" – residual plot da konus shakli

4. Normallik:

Qoldiqlar normal taqsimlangan bo'lishi kerak

(katta n da bu shart unchalik muhim emas – CLT tufayli)

5. Multikollinearlik yo'qligi (ko'p o'zgaruvchili holatda):
X o'zgaruvchilar bir-biri bilan kuchli korrelyatsiyada bo'lmasin
Tekshirish → VIF (Variance Inflation Factor)
VIF > 10 → muammo bor
-

8. Sklearn da Amaliyot

```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score, mean_squared_error
import numpy as np
```

```
# Ma'lumotni train/test ga bo'lish
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,          # 20% test
    random_state=42         # takrorlanuvchi natija
)
```

```
# Model yaratish va o'rgatish
```

```
model = LinearRegression()
model.fit(X_train, y_train)
```

```
# Koeffitsiyentlar
```

```
print(f"Intercept ( $\beta_0$ ): {model.intercept_:.2f}")
print(f"Slope ( $\beta_1$ ): {model.coef_[0]:.2f}")
```

```
# Bashorat
```

```
y_pred = model.predict(X_test)
```

```
# Baholash
```

```
r2 = r2_score(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
```

```
print(f"R2: {r2:.4f}")      # 0.85 → 85% tushuntiradi
print(f"MSE: {mse:.2f}")
print(f"RMSE: {rmse:.2f}")  # Y bilan bir xil birlik
```

Train/Test ajratish nima uchun kerak:

Model o'rgangan ma'lumotda yaxshi ishlashi tabiiy –
u o'sha ma'lumotni "yodlab" olgan bo'lishi mumkin.

Haqiqiy sinov: model ko'rmagan ma'lumotda qanday ishlaydi?

Train set → model o'rganadi (β_0 , β_1 topiladi)

Test set → model baholanadi (haqiqiy sifat o'lchanadi)

Overfitting:

Train $R^2 = 0.99$, Test $R^2 = 0.45$ → Model yodlagan, o'rganmagan!

9. Xulosa

Chiziqli regressiya:

$$Y = \beta_0 + \beta_1 X + \varepsilon$$

OLS maqsadi:

$$\sum (Y_i - \hat{Y}_i)^2 \rightarrow \text{minimal}$$

Koeffitsiyentlar:

$\beta_0 \rightarrow X=0$ da Y qiymati (ko'p holat mantiqsiz)

$\beta_1 \rightarrow X$ 1 birlikka oshsa Y β_1 ga o'zgaradi

Baholash metrikalari:

$R^2 \rightarrow$ O'zgarishning qancha % tushuntirildi (0-1)

MSE $\rightarrow$ O'rtacha kvadratik xato (katta xatolarga sezgir)

RMSE $\rightarrow$ MSE ildizi, Y bilan bir xil birlik

MAE $\rightarrow$ O'rtacha mutlaq xato, outlierga bardoshli

Muhim shart:

Train/Test ajratish $\rightarrow$ overfittingni aniqlash

Adjusted $R^2 \rightarrow$ ko'p o'zgaruvchili holat uchun

💡 **Eslab qol:** R^2 yuqori bo'lishi yaxshi model degani emas. **Test to'plamida** R^2 yuqori bo'lishi — yaxshi model belgisi. Train da $R^2=0.99$, test da $R^2=0.5 \rightarrow$ model yodlagan, bashorat qila olmaydi!

● SAVOL 23:

LOGISTIK REGRESSIYA KLASSEFIKATSIYA UCHUN QANDAY ISHLAYDI? SIGMOID FUNKSIYASI, IKKILIK CHIQISH VA EHTIMOLLIK BAHOSI. NIMA UCHUN U LINEAR REGRESSION EMAS?

1. Muammo — Nima uchun Chiziqli Regressiya Klassifikatsiya Uchun Yaramaydi?

Klassifikatsiya — raqam emas, **kategoriya** bashorat qilish:

Regressiya: $Y = 25,000$ so'm, 37.5 daraja, 182.3 sm

Klassifikatsiya: $Y =$ "Ha/Yo'q", "Spam/Spam emas", "Kasal/Sog' "

Chiziqli regressiyani klassifikatsiyaga qo'llashga urinish:

$Y = 0$ (Yo'q) yoki 1 (Ha) deb belgilaymiz

Chiziqli regressiya: $\hat{Y} = \beta_0 + \beta_1 X$

Muammo 1 — Chegara muammosi:

X katta bo'lsa $\rightarrow \hat{Y} = 2.7$ (1 dan katta — ehtimollik bo'la olmaydi!)

X kichik bo'lsa $\rightarrow \hat{Y} = -0.4$ (0 dan kichik — ehtimollik bo'la olmaydi!)

Ehtimollik: faqat $[0, 1]$ oralig'ida bo'lishi kerak!

Muammo 2 — Chiziqlilik muammosi:

"Kasal" ehtimolligi dori miqdoriga chiziqli bog'liq emas

Aslida: kam dozada — ta'sir yo'q, ma'lum darajadan keyin

tez oshadi, yuqori dozada – to'yinish (1 ga yaqinlashadi)
Bu S-shakl – chiziqli emas!

Muammo 3 – Yangi nuqtalar chiziqni "tortib" ketadi:
Bir nechta aniq 1 lardan uzoq nuqta qo'shilsa →
regressiya chizig'i siljiydi → klassifikatsiya buziladi

2. Sigmoid Funksiyasi — Yechim

Sigmoid (S-shakl) funksiyasi istalgan haqiqiy sonni **(0, 1)** oralig'iga siqib qo'yadi:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

bu yerda:

$z = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots$ (chiziqli kombinatsiya)
 $e \approx 2.718$ (Eyler soni)

Sigmoid xususiyatlari:

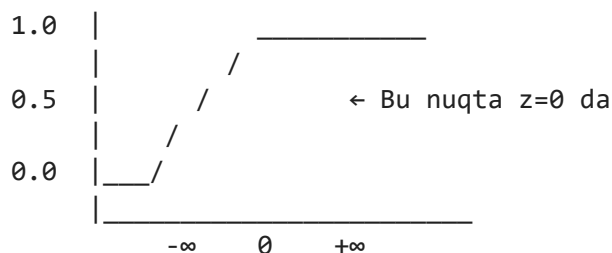
$z \rightarrow -\infty$: $\sigma(z) \rightarrow 0$ (hech qachon 0 ga teng emas)
 $z = 0$: $\sigma(z) = 0.5$ (aniq o'rta nuqta)
 $z \rightarrow +\infty$: $\sigma(z) \rightarrow 1$ (hech qachon 1 ga teng emas)

Simmetriya: $\sigma(-z) = 1 - \sigma(z)$

Differensiallash (gradient descent uchun muhim):

$\sigma'(z) = \sigma(z) \cdot (1 - \sigma(z))$
→ O'zining qiymati orqali hisoblanadi – juda qulay!

Sigmoid shakli:



3. Logistik Regressiya Qanday Ishlaydi?

2 bosqich:

Bosqich 1 – Chiziqli kombinatsiya:

$z = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_n X_n$

Bosqich 2 – Sigmoid orqali ehtimollikka aylantirish:

$$P(Y=1) = \frac{1}{1 + e^{-z}}$$

$$1+e^{-z}$$

Natija:

$P(Y=1) = 0.87 \rightarrow 87\%$ ehtimollik bilan sinf 1

$P(Y=0) = 0.13 \rightarrow 13\%$ ehtimollik bilan sinf 0

$(P(Y=0) = 1 - P(Y=1) - \text{ikkilik holat})$

Qaror chegara (Decision Boundary):

Standart: $P(Y=1) \geq 0.5 \rightarrow \text{Sinf 1}$

$P(Y=1) < 0.5 \rightarrow \text{Sinf 0}$

$P = 0.5 \rightarrow z = 0 \rightarrow \beta_0 + \beta_1 X = 0 \rightarrow$ bu qaror chegara chizig'i

Chegara o'zgartirish mumkin:

Tibbiyotda: $P \geq 0.3 \rightarrow \text{"Kasal"}$ (xato o'tkazib yubormaslik muhim)

Spam: $P \geq 0.7 \rightarrow \text{"Spam"}$ (muhim xat spam deb belgilanmasin)

Chegara tanlash $\rightarrow$ precision va recall o'rtasidagi muvozanat

4. Odds va Log-Odds — Matematik Asos

Logistik regressiyani chuqur tushunish uchun **odds** tushunchasi kerak:

Ehtimollik (Probability):

$P = 0.8 \rightarrow \text{"80\% ehtimollik bilan sodir bo'ladi"}$

Odds (Nisbat):

$\text{Odds} = P / (1-P)$

$P=0.8 \rightarrow \text{Odds} = 0.8/0.2 = 4$

Ma'nosi: "4:1 nisbatda sodir bo'ladi"

Log-Odds (Logit):

$\text{logit}(P) = \ln(\text{Odds}) = \ln(P / (1-P))$

$P=0.5 \rightarrow \text{Odds}=1 \rightarrow \text{log-odds}=0$

$P=0.8 \rightarrow \text{Odds}=4 \rightarrow \text{log-odds}=1.39$

$P=0.95 \rightarrow \text{Odds}=19 \rightarrow \text{log-odds}=2.94$

$P=0.2 \rightarrow \text{Odds}=0.25 \rightarrow \text{log-odds}=-1.39$

Logistik regressiyaning haqiqiy shakli:

Logistik regressiya aslida log-odds ni chiziqli modellashtiradi:

$$\ln(P/(1-P)) = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots$$

Bu aylantirilganda $\rightarrow$ Sigmoid formulasiga keladi!

Demak:

Chiziqli regressiya: Y ni to'g'ridan-to'g'ri modellashtiradi

Logistik regressiya: log-odds ni chiziqli modellashtiradi,
keyin sigmoid orqali ehtimollikka aylantiradi

5. Koeffitsiyentlarni Talqin Etish

Chiziqli regressiyadan farqli — logistik regressiya koeffitsiyentlari **odds ratio** orqali talqin etiladi:

Chiziqli regressiya:

$\beta_1 = 500 \rightarrow$ "X 1 ga oshsa Y 500 ga oshadi"

Oddiy, to'g'ridan-to'g'ri talqin

Logistik regressiya:

β_1 ning to'g'ridan-to'g'ri talqini qiyin

(log-odds o'zgarishi — intuitiv emas)

Odds Ratio orqali talqin:

$OR = e^{\beta_1}$

$\beta_1 = 0.7 \rightarrow OR = e^{0.7} = 2.01$

Talqin: "X 1 ga oshsa, odds 2 marta oshadi"

$\beta_1 = -0.5 \rightarrow OR = e^{(-0.5)} = 0.61$

Talqin: "X 1 ga oshsa, odds 39% kamayadi"

$\beta_1 = 0 \rightarrow OR = 1$

Talqin: "X ning hech qanday ta'siri yo'q"

6. Model O'rgatish — Maximum Likelihood

Chiziqli regressiya OLS (kvadratlar yig'indisini minimallashtirish) ishlatadi. Logistik regressiya esa **Maximum Likelihood Estimation (MLE)** ishlatadi.

OLS nima uchun yaramaydi:

$Y \in \{0, 1\} \rightarrow$ kvadratik xato funksiyasi notekis (non-convex)

$\rightarrow$ Gradient descent lokal minimumga tushib qoladi

MLE g'oyasi:

"Model parametrlarini shunday tanlayki, kuzatilgan

ma'lumotlarning ehtimolligi maksimal bo'lsin"

Log-Loss (Binary Cross-Entropy):

$$L = -1/n \cdot \sum [Y_i \cdot \ln(\hat{Y}_i) + (1 - Y_i) \cdot \ln(1 - \hat{Y}_i)]$$

$Y_i=1$ holat: $-\ln(\hat{Y}_i) \rightarrow \hat{Y}=0.9 \rightarrow$ xato kichik

$\rightarrow \hat{Y}=0.1 \rightarrow$ xato KATTA

$Y_i=0$ holat: $-\ln(1 - \hat{Y}_i) \rightarrow \hat{Y}=0.1 \rightarrow$ xato kichik

$\rightarrow \hat{Y}=0.9 \rightarrow$ xato KATTA

Log funksiyasi: noto'g'ri ishonchli bashoratga

katta jarima beradi $\rightarrow$ model ehtiyotkor bo'ladi

7. Baholash Metrikalari

R^2 va MSE klassifikatsiya uchun mos emas. Buning o'rniga:

Confusion Matrix:

	Bashorat: 0	Bashorat: 1
Haqiqiy: 0	TN	FP
Haqiqiy: 1	FN	TP

TP = True Positive (to'g'ri 1)

TN = True Negative (to'g'ri 0)

FP = False Positive (noto'g'ri 1 – I tur xato)

FN = False Negative (noto'g'ri 0 – II tur xato)

Accuracy (To'g'rilik):

$(TP + TN) / (TP + TN + FP + FN)$

Muammo: Nomutanosib klasslarda yolg'on yuqori ko'rinadi
(99% kasal emas → hammani "sog'" desam → 99% accuracy!)

Precision (Aniqlik):

$TP / (TP + FP)$

"1 deb aytganimning qanchasi haqiqatan 1?"

Recall (Qamrov):

$TP / (TP + FN)$

"Barcha 1 larning qanchasini topdim?"

F1 Score:

$2 \cdot (Precision \cdot Recall) / (Precision + Recall)$

Precision va Recall o'rtasidagi muvozanat

ROC-AUC:

Turli chegara qiymatlarida model sifati

AUC = 1.0 → mukammal, AUC = 0.5 → tasodifiy

8. Sklearn da Amaliyot

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (classification_report,
                             confusion_matrix, roc_auc_score)
```

```
# Model yaratish
```

```
model = LogisticRegression(
    C=1.0,                      # Regularizatsiya kuchi (1/λ)
    max_iter=1000,              # Iteratsiyalar soni
    random_state=42
)
```

```
model.fit(X_train, y_train)
```

```
# Ehtimollik bashorati
```

```
ehtimollik = model.predict_proba(X_test)
# [[0.13, 0.87], ← 1-kuzatuv: 87% sinf 1
# [0.91, 0.09], ← 2-kuzatuv: 91% sinf 0
# ...]
```

```
# Sinf bashorati (chegara=0.5)
```

```
y_pred = model.predict(X_test)
```

Baholash

```
print(classification_report(y_test, y_pred))  
print(f"AUC: {roc_auc_score(y_test, ehtimollik[:,1]):.4f}")
```

Regularizatsiya — muhim tushuncha:

Muammo: Ko'p o'zgaruvchi → model haddan oshadi (overfit)
Yechim: Koeffitsiyentlarni kichik bo'lishga "majbur" qilish

L2 (Ridge) – standart:

Katta koeffitsiyentlarga jarima
Barcha o'zgaruvchilar qoladi, lekin kichiklashadi

L1 (Lasso):

Ba'zi koeffitsiyentlarni aynan 0 ga tushiradi
→ Avtomatik feature tanlash

C parametri:

C kichik → Kuchli regularizatsiya → sodda model
C katta → Zaif regularizatsiya → murakkab model

9. Chiziqli va Logistik Regressiya Taqqoslash

Xususiyat	Chiziqli	Logistik
Maqsad	Raqam bashorat	Sinf bashorat
Chiqish	$-\infty$ dan $+\infty$ gacha	0 dan 1 gacha
Chiqish ma'nosi	Qiyamat	Ehtimollik
O'rgatish usuli	OLS (analitik)	MLE (iterativ)
Xato funksiyasi	MSE	Log-Loss
Baholash	R^2 , RMSE	Accuracy, F1, AUC
Taxmin	Y chiziqli	Log-odds chiziqli

10. Xulosa

Logistik regressiya:

- $z = \beta_0 + \beta_1 X$ (chiziqli kombinatsiya)
- $P = \text{sigmoid}(z) = 1/(1+e^{-z})$ (ehtimollikka aylantirish)
- $P \geq 0.5 \rightarrow \text{Sinf } 1, P < 0.5 \rightarrow \text{Sinf } 0$

Sigmoid:

- Istalgan sonni (0,1) ga siqadi
- S-shakl → to'yinish (saturation) bor
- $z=0$ da $P=0.5$ – qaror chegarasi

Nima uchun chiziqli emas:

- Chiziqli chiqish [0,1] dan chiqadi
- Ehtimollik emas, raqam chiqaradi
- Outlierlar chegarani buzadi

Baholash:

- Precision → noto'g'ri musbat xatosi muhim bo'lsa
- Recall → noto'g'ri manfiy xatosi muhim bo'lsa

F1 → ikkalasi muvozanati kerak bo'lsa
AUC → umumiy model sifati

💡 **Eslab qol:** Logistik regressiya — nomi "regressiya" bo'lsa ham, aslida **klassifikatsiya** algoritmi. U ehtimollik chiqaradi — bu uning eng kuchli tomoni. "Bu bemor 73% ehtimollik bilan kasal" — bu "Ha/Yo'q" dan ko'ra ancha foydali ma'lumot!

● SAVOL 24:

K-MEANS KLASTERIZATSIYA ALGORITMINING ISHLASH PRINSIPI: SENTROIDLAR, ITERATSIYA VA KONVERGENSIYA. **K** SONI QANDAY TANLANADI? **ELBOW METHOD** NIMA?

1. Klasterizatsiya — Nazariy Asos

Klasterizatsiya — **o'qituvchisiz o'rganish (unsupervised learning)** turi. Regressiya va klassifikatsiyadan farqi:

Regressiya/Klassifikatsiya (Supervised):

Ma'lumot: X (belgilar) + Y (javob) → Model Y ni bashorat qiladi

Misol: 1000 ta uy: maydon, narx berilgan → yangi uyning narxini top

Klasterizatsiya (Unsupervised):

Ma'lumot: faqat X (belgilar), Y YO'Q → Model guruhlarini o'zi topadi

Misol: 1000 ta mijoz: yoshi, xarid miqdori → o'xshashlarni guruhlash

Klasterizatsiya savoli:

"Bu ma'lumotlarda qanday tabiiy guruhlar yashiringan?"

Qachon ishlatiladi:

Mijozlarni segmentlash	→ Marketing strategiyasi
Genllarni guruhlash	→ Biologiya tadqiqoti
Hujjatlarni guruhlash	→ Axborot qidirish
Anomaliya aniqlash	→ Kiberhavfsizlik
Rasm siqish	→ Ranglarni kamaytirish
Tavsiya tizimlari	→ O'xshash foydalanuvchilar

2. K-Means — Asosiy G'oya

K-Means — ma'lumotlarni **K ta klasterga** bo'lish algoritmi. Har klaster **sentroid** (markaziy nuqta) atrofida joylashgan nuqtalardan iborat.

Intuitiv tushuncha:

Tasavvur qiling: Xaritada 100 ta shahar bor.

$K=3$ ta omborxona qurmoqchisiz.

Maqsad: Har shahar o'ziga eng yaqin omborxonaga tegishli bo'lsin

va umumiy masofa minimall bo'lsin.

K-Means aynan shu masalani yechadi!
Omborxona joylashuvi → Sentroid
Shaharlar → Ma'lumot nuqtalari

Maqsad funksiyasi (minimallashtirish):

$$WCSS = \sum_k \sum_{x_i \in C_k} ||x_i - \mu_k||^2$$

WCSS – Within-Cluster Sum of Squares

μ_k – k-chi klasterning sentroidi

x_i – k-chi klasterdagi nuqta

Ma'nosi: Har nuqtaning o'z sentroididan kvadratik masofalar yig'indisi → bu minimal bo'lsin

3. Algoritm Bosqichlari — Iteratsiya

Bosqich 1 – Inisializatsiya:

K ta sentroidni TASODIFIY tanlash
(yoki K-Means++ usuli bilan – quyida tushuntiriladi)

Bosqich 2 – Tayinlash (Assignment):

Har nuqtani ENG YAQIN sentroidga tayinlash
 $d(x, \mu_k) = \sqrt{\sum (x_i - \mu_{k_i})^2} \leftarrow$ Evklid masofasi

Bosqich 3 – Yangilash (Update):

Har klasterning yangi sentroidini hisoblash

$$\mu_k = (1/|C_k|) \cdot \sum_{x_i \in C_k} x_i$$

Ya'ni: klasterdagi barcha nuqtalarning o'rtachasi

Bosqich 4 – Konvergensiya tekshiruvi:

Sentroidlar o'zgardimi?

Ha → Bosqich 2 ga qayt

Yo'q → TUGADI (konvergensiya)

Vizual ko'rinish:

Boshlang'ich:

```
* * * x * *
* *   x   *
* *x     * *
  * x * *
x → tasodifiy
   sentroid
```

Tayinlash:

```
* 1 * 1 * 1 x * 2 * 2
* 1 * 1   x 1   * 2
* 1 * 1 x 1   * 2 * 2
      * 3 x 3 * 3 * 2
x → sentroid
1,2,3 → klaster raqami
```

Yangilash:

```
* 1 * 1 * 1 • * 2 * 2
* 1 * 1   •   * 2
* 1 * 1 •     * 2 * 2
      * 3 • * 3 * 2
• → yangi sentroid
```

Iteratsiya davom etadi... → Sentroidlar barqarorlashadi → KONVERGENSIYA

4. Konvergensiya — Nazariy Kafolat

K-Means HAR DOIM konvergensiya erishadi.
Nima uchun?

- Har iteratsiyada WCSS kamayadi yoki o'zgarmaydi
- WCSS pastdan cheklangan (0 dan kichik bo'la olmaydi)
- Cheklangan sonli nuqtalar → cheklangan sonli tayinlash kombinatsiyasi
- Demak: Chekli sonli iteratsiyada WCSS minimal bo'ladi

Lekin: Global minimum emas, LOCAL minimum!

- K-Means boshlang'ich sentroidlarga bog'liq →
- Har xil boshlang'ich → Har xil natija mumkin!

Lokal minimum muammosi:

Tasodifiy inisializatsiya:

- Yaxshi boshlang'ich → Yaxshi klasterlar 
- Yomon boshlang'ich → Yomon klasterlar 

Yechim 1 — Ko'p marta ishlatish:

```
model = KMeans(n_init=10) ← 10 marta ishlatib, eng yaxshini tanlaydi
```

Yechim 2 — K-Means++ (zamonaviy standart):

Sentroidlarni "aqlli" tanlash

5. K-Means++ — Aqlli Inisializatsiya

Oddiy K-Means tasodifiy sentroid tanlaydi → yomon natija mumkin. K-Means++ bu muammoni yechadi:

K-Means++ algoritmi:

1. Birinchi sentroidni tasodifiy tanlash
2. Keyingi sentroid uchun:
 - Har nuqtaning eng yaqin sentroiddan masofasini hisoblash
 - Masofa katta bo'lgan nuqtalar ko'proq ehtimollik bilan tanlanadi

$$P(x \text{ tanlash}) = d(x)^2 / \sum d(x_i)^2$$

3. K ta sentroid to'lguncha 2-bosqichni takrorlash

Natija:

- Sentroidlar bir-biridan uzoq joylashadi
- Klasterlar butun fazoni yaxshiroq qamrab oladi
- Lokal minimum ehtimolligi kamayadi
- Tezroq konvergensiya

sklearn da K-Means++ standart:

```
from sklearn.cluster import KMeans
```

```
model = KMeans(  
    n_clusters=3,
```

```

init="k-means++", # ← standart, K-Means++ ishlatadi
n_init=10,        # 10 marta ishlatib, eng yaxshini tanlaydi
max_iter=300,     # maksimal iteratsiya soni
random_state=42
)

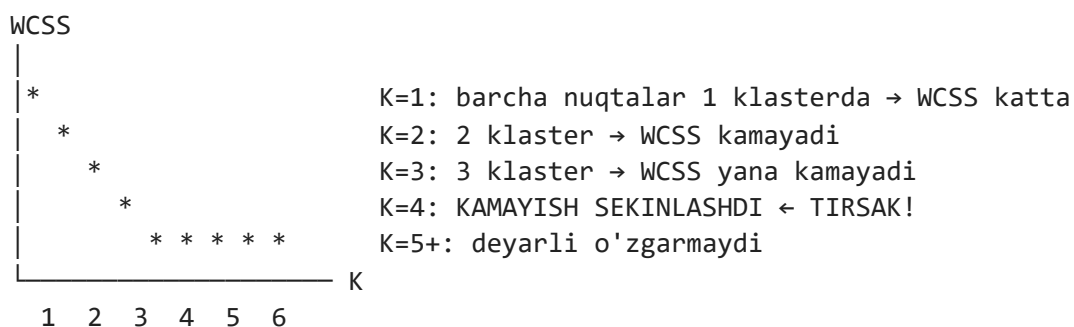
```

6. K Soni Qanday Tanlanadi?

Bu K-Means ning **eng muhim va qiyin** savoli. K oldindan ma'lum bo'lmaydi.

Elbow Method — Tirsak Usuli:

Nazariy asos: K oshgan sari WCSS (Within-Cluster Sum of Squares) kamayadi. Lekin ma'lum K dan keyin kamayish sekinlashadi. Bu "tirsak" nuqtasi optimal K.



Optimal K = 4 (tirsakning "boshi")

Nima uchun eng katta K tanlamaslik kerak:

K = n (har nuqta o'z klasterida) → WCSS = 0 → Befoyda!
 Klaster maqsadi: ma'lumotdagi TABIIY guruhlarini topish,
 sun'iy bo'lish emas.

Tirsak: "Ko'proq klaster qo'shish sezilarli foyda bermayapti"
 degan signaldir.

```

wcss = []
K_oraliq = range(1, 11)

for k in K_oraliq:
    model = KMeans(n_clusters=k, init="k-means++",
                   n_init=10, random_state=42)
    model.fit(X)
    wcss.append(model.inertia_) # inertia_ = WCSS

# Grafik
plt.plot(K_oraliq, wcss, "bo-", markersize=8)
plt.xlabel("K (klasterlar soni)")
plt.ylabel("WCSS")
plt.title("Elbow Method")
plt.xticks(K_oraliq)
plt.grid(True, alpha=0.3)
plt.show()

```

7. Boshqa K Tanlash Usullari

Elbow method har doim aniq javob bermaydi. Qo'shimcha usullar:

Silhouette Score:

Har nuqta uchun:

a = O'z klasteridagi boshqa nuqtalarga o'rtacha masofa

b = Eng yaqin boshqa klasterdagi nuqtalarga o'rtacha masofa

$s = (b - a) / \max(a, b)$

s → +1 : Nuqta o'z klasterida juda yaxshi joylashgan

s → 0 : Nuqta chegarada

s → -1 : Nuqta noto'g'ri klasterda!

O'rtacha Silhouette Score → yuqori bo'lsa yaxshi

Optimal K → Silhouette Score maksimal bo'lgan K

```
from sklearn.metrics import silhouette_score
```

```
silhouette_scores = []
```

```
for k in range(2, 11): # k=1 uchun hisoblab bo'lmaydi
```

```
    model = KMeans(n_clusters=k, random_state=42, n_init=10)
```

```
    labels = model.fit_predict(X)
```

```
    skor = silhouette_score(X, labels)
```

```
    silhouette_scores.append(skor)
```

Gap Statistic:

G'oya: Haqiqiy ma'lumotdagi WCSS ni

tasodifiy ma'lumotdagi WCSS bilan solishtirish

$Gap(K) = E[\log(WCSS_{tasodifiy})] - \log(WCSS_{haqiqiy})$

Gap katta → Haqiqiy klaster tuzilishi bor

Gap kichik → Tasodifiy taqsimotdan farq yo'q

Optimal K → Gap eng katta bo'lgan K

8. K-Means ning Cheklovlari

Cheklov 1 – Doira shaklidagi klasterlar:

K-Means Evklid masofasiga asoslangan →

Faqat sferik (doira, shar) shaklni yaxshi topadi

Egri, uzunchoq, halqa shakldagi klasterlar → yomon natija

Yechim: DBSCAN, Gaussian Mixture Models

Cheklov 2 – Bir xil o'lchamli klasterlar:

K-Means klasterlarni teng o'lchamda kutadi

Bitta katta + bitta kichik klaster → xato bo'linishi mumkin

Cheklov 3 – Outlier ta'siri:

Sentroid = o'rtacha → Outlierlar sentroidni "tortib" ketadi

Yechim: K-Medoids (sentroid o'rniga haqiqiy nuqta)

Cheklov 4 – K oldindan belgilanishi:

Haqiqiy dunyo ma'lumotida K noma'lum → Elbow/Silhouette kerak

Cheklov 5 – Miqyos sezgirligi:

Katta o'lchamli o'zgaruvchilar masofa hisoblashda dominant bo'ladi

Yechim: Standartlashtirish (StandardScaler) → SHART!

9. Standartlashtirish — Nima Uchun Shart?

Misol: Mijozlarni klasterlash

Yosh: 20-60 yil → o'rtacha 40, tarqoqlik 10

Daromad: 500-50000 so'm → o'rtacha 10000, tarqoqlik 5000

Masofa hisoblashda:

$d = \sqrt{(\text{yosh farqi})^2 + (\text{daromad farqi})^2}$

$d = \sqrt{(2)^2 + (3000)^2} \approx 3000$

Daromad masofasi yosh masofasini "bosib ketadi"!

Model aslida faqat daromad bo'yicha klasterlaydi.

Yechim – StandardScaler:

Har o'zgaruvchi: $(X - \mu) / \sigma$

Natija: Barcha o'zgaruvchilar bir xil miqyosda (o'rtacha 0, $\sigma=1$)

Endi: Yosh va daromad teng hissa qo'shadi

```
from sklearn.preprocessing import StandardScaler
```

```
scaler = StandardScaler()
```

```
X_scaled = scaler.fit_transform(X)
```

```
model = KMeans(n_clusters=3, random_state=42, n_init=10)
```

```
model.fit(X_scaled)
```

10. Xulosa

K-Means algoritmi:

1. K ta sentroid tasodifiy (yoki K-Means++ bilan) tanlash
2. Har nuqtani eng yaqin sentroidga tayinlash
3. Klaster o'rtachasini yangi sentroid sifatida hisoblash
4. Sentroidlar o'zgarmaguncha 2-3 takrorlash → Konvergeniya

Konvergeniya:

→ Har doim yetadi (WCSS monoton kamayadi)

→ Lekin lokal minimum (global emas) → `n_init=10` yechim

K tanlash:

Elbow Method → WCSS grafikida "tirsak" nuqtasi
Silhouette → Nuqtaning o'z klasteriga "moslik" darajasi
Gap Statistic → Tasodifiy vs haqiqiy taqsimot taqqoslash

Muhim amaliy qoidalar:

- ✓ StandardScaler → klasterlashdan OLDIN shart
- ✓ K-Means++ (init="k-means++") → standart ishlatish
- ✓ n_init=10 → lokal minimum dan himoya
- ✓ Elbow + Silhouette → ikkalasini birga ishlatish
- ✓ Natijani vizualizatsiya → klasterlar mantiqliligini tekshirish

💡 **Eslab qol:** K-Means har doim K ta klaster topadi — ma'lumotda haqiqiy K ta guruh bo'lmasa ham! Shuning uchun natijani har doim **domain knowledge** (soha bilimlari) bilan tekshirish kerak. Algoritm matematik optimum topadi, lekin u biologik yoki biznes ma'nosiga ega ekanligini siz tekshirasiz!

● SAVOL 25:

TRENING VA TEST TO'PLAMIGA BO'LISH (TRAIN-TEST SPLIT) NIMA UCHUN KERAK?
OVERFITTING VA UNDERFITTING TUSHUNCHALARI, ULARNI ANIQLASH VA OLDINI OLISH USULLARI.

1. Train-Test Split — Nima Uchun Kerak?

Asosiy muammo: Model o'rgangan ma'lumotda yaxshi ishlashi — bu uning haqiqiy qobiliyatini ko'rsatmaydi. Model ma'lumotni shunchaki **yodlab** olgan bo'lishi mumkin.

Analogiya – Imtihon:

O'quvchi faqat o'tgan yilgi savollarni yodlaydi
O'tgan yil savollari → 100% javob beradi
Yangi savollar → hech narsa bilmaydi

Bu "o'rganish" emas – "yodlash"!

Modelda xuddi shu muammo:

Train ma'lumotida → $R^2=0.99$ (yodlab olgan)
Yangi ma'lumotda → $R^2=0.45$ (haqiqiy bilim past)

Yechim — Ma'lumotni ajratish:

Barcha ma'lumot (100%)

↓

Train (80%) Model shu yerda o'rgan.	Test(20%) Model bu yerda baholanadi
---	--

Qoida: Test to'plami MODEL UCHUN "Ko'RINMAS" bo'lishi kerak!
Train jarayonida test ma'lumoti hech qachon ishlatilmaydi.

2. Train-Validation-Test — Uch Qismga Bo'lish

Amalda ikki qismga emas, **uch qismga** bo'lish professional yondashuv:

Barcha ma'lumot (100%)

↓

Train (60%) O'rgatish	Val (20%) Giperpar. tanlash	Test(20%) Yakuniy baho
--------------------------	-----------------------------------	------------------------------

Train → Model o'rganadi (β , w parametrlar topiladi)
Validation → Giperparametrlar tanlanadi ($K=?$, $depth=?$, $C=?$)
Test → Bir marta, yakuniy baholash uchun

Nima uchun Validation kerak?

Agar giperparametrlarni test da tanlasak →

Test ham "ko'rilgan" bo'lib qoladi → natija ishonchsiz!

3. Cross-Validation — Kuchli Baholash Usuli

Kichik ma'lumot to'plamida 80/20 bo'lish ishonchsiz — test to'plami "omadli" yoki "omadsiz" bo'lishi mumkin. **K-Fold Cross-Validation** bu muammoni hal qiladi:

5-Fold Cross-Validation:

Ma'lumot: [1 | 2 | 3 | 4 | 5]

Iteratsiya 1: Train[2,3,4,5] Test[1] → Score₁

Iteratsiya 2: Train[1,3,4,5] Test[2] → Score₂

Iteratsiya 3: Train[1,2,4,5] Test[3] → Score₃

Iteratsiya 4: Train[1,2,3,5] Test[4] → Score₄

Iteratsiya 5: Train[1,2,3,4] Test[5] → Score₅

Final Score = (Score₁+Score₂+Score₃+Score₄+Score₅) / 5

Afzalligi:

→ Har nuqta bir marta test, to'rt marta train da ishlatiladi

→ Barcha ma'lumot baholashga hissa qo'shadi

→ Natija barqarorroq va ishonchli

```
from sklearn.model_selection import cross_val_score, KFold
```

```
kf = KFold(n_splits=5, shuffle=True, random_state=42)
```

```
skorlar = cross_val_score(model, X, y, cv=kf, scoring="r2")
```

```
print(f"Har fold skori: {skorlar}")
```

```
print(f"O'rtacha: {skorlar.mean():.4f} ± {skorlar.std():.4f}")  
# ± qiymati kichik bo'lsa → model barqaror
```

4. Overfitting — Nazariy Asos

Overfitting — model train ma'lumotini juda yaxshi o'rganadi, lekin yangi ma'lumotda yomon ishlaydi.
Model **shovqinni ham o'rganib oladi**.

Analogiya:

Talaba darslik savollarini yod oladi (shu jumladan xato bosmalarniyam)
Imtihonda yangi shakldagi savol → javob bera olmaydi

Model nuqtai nazaridan:

Ma'lumot = Haqiqiy naqsh + Shovqin (tasodifiy xatolar)

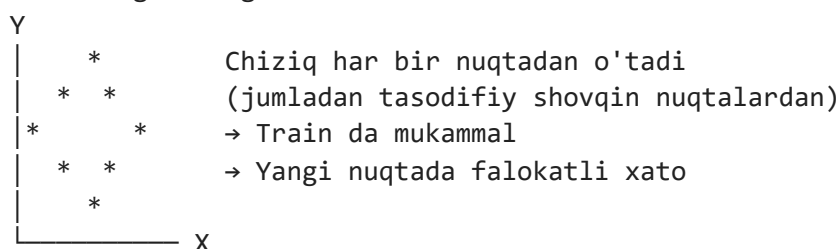
Underfitting: Model haqiqiy naqshni ham o'rgana olmaydi

Optimal: Model haqiqiy naqshni o'rganadi, shovqinni emas

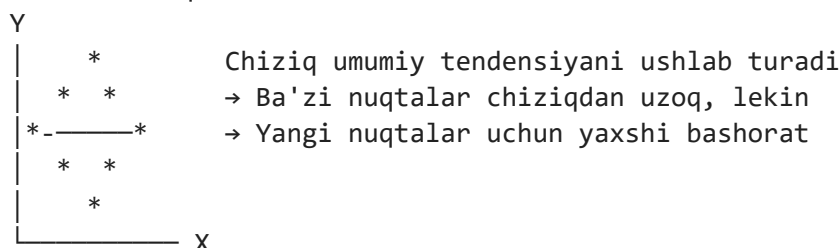
Overfitting: Model haqiqiy naqsh + shovqinni ham o'rganadi

Grafik ko'rinishda:

Overfitting chizig'i:



Optimal chiziq:



Overfitting belgilari:

Train $R^2 = 0.98$, Test $R^2 = 0.52$ ← Katta farq → Overfitting!

Train MSE = 12, Test MSE = 340 ← Katta farq → Overfitting!

Qoida: Train va Test skarlari orasidagi katta farq → Overfitting

Overfitting sabablari:

1. Model juda murakkab:

- Ko'p parametrlı neyron tarmoq
- Juda chuqur qaror daraxti
- Ko'p darajali polinom

2. Ma'lumot juda kam:
→ 10 ta kuzatuv, 8 ta o'zgaruvchi → parametrlar ko'p!
→ Model "yodlaydi"
 3. O'rgatish juda uzoq (neyron tarmoqda):
→ Iteratsiya ko'p → model shovqinni o'rganib oladi
 4. Regularizatsiya yo'q:
→ Koeffitsiyentlarga hech qanday cheklov yo'q
-

5. Underfitting — Nazariy Asos

Underfitting — model juda sodda, ma'lumotdagi haqiqiy naqshni ham ushlab ololmaydi.

Analogiya:

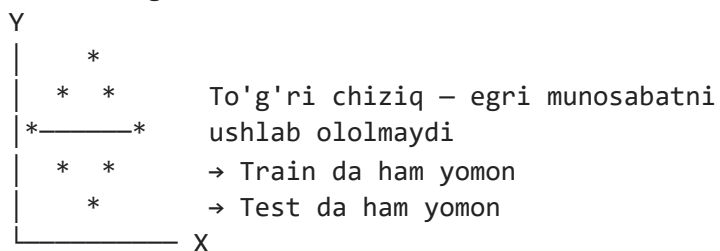
Talaba darslikni umuman o'qimaydi →
Eng oddiy savolga ham javob bera olmaydi

Model nuqtai nazaridan:

Chiziqli model → Egri munosabatni tushuntir olmaydi
Juda kichik neyron tarmoq → Murakkab naqshni o'rgana olmaydi

Grafik ko'rinishda:

Underfitting:



Underfitting belgilari:

Train $R^2 = 0.45$, Test $R^2 = 0.43$ ← Ikkalasi ham past
(Farq kichik, lekin ikkala natija ham yomon)

Underfitting sabablari:

1. Model juda sodda:
→ Chiziqli model, lekin munosabat egri
→ Kam parametrlar model
 2. O'rgatish juda qisqa (neyron tarmoqda):
→ Iteratsiya kam → model to'liq o'rgana olmagan
 3. Juda kuchli regularizatsiya:
→ Koeffitsiyentlar juda kichraytirilgan
→ Model deyarli hech narsa o'rgana olmaydi
 4. Muhim o'zgaruvchilar olib tashlangan:
→ Feature engineering yetarli emas
-

6. Bias-Variance Tradeoff — Fundamental Nazariya

Overfitting va underfitting ning matematik asosi:

$$\text{Model xatosi} = \text{Bias}^2 + \text{Variance} + \text{Qaytarilmas Shovqin}$$

Bias (Sistemli xato):

- Model qanchalik "noto'g'ri farazlaydi"
- Underfitting → Yuqori Bias
- Chiziqli model egri munosabatni o'rgana olmaydi

Variance (Beqarorlik):

- Model turli train to'plamlarida qanchalik farq qiladi
- Overfitting → Yuqori Variance
- Har yangi ma'lumotda model keskin o'zgaradi

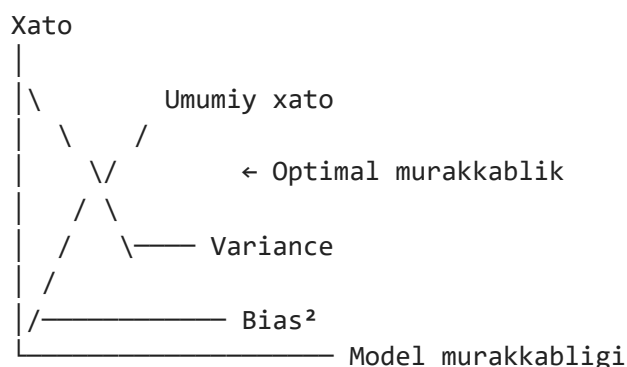
Qaytarilmas Shovqin:

- Ma'lumotdagi tabiiy tasodifiylik
- Hech qanday model buni bartaraf eta olmaydi

Tradeoff:

- Murakkablik ↑ → Bias ↓, Variance ↑
- Murakkablik ↓ → Bias ↑, Variance ↓

Maqsad: Ikkalasi ham minimal bo'lgan nuqta → Optimal model



7. Overfitting Oldini Olish Usullari

Usul 1 — Regularizatsiya:

G'oya: Koeffitsiyentlar juda katta bo'lmazligiga jarima berish
Katta koeffitsiyent → Model shovqinga haddan oshiqcha ta'sirchan

L2 (Ridge):

- $\text{Xato} = \text{MSE} + \lambda \cdot \sum \beta_i^2$
- Koeffitsiyentlarni kichraytiradi, 0 ga tushirmaydi
- Barcha o'zgaruvchilar qoladi

L1 (Lasso):

- $\text{Xato} = \text{MSE} + \lambda \cdot \sum |\beta_i|$
- Ba'zi koeffitsiyentlarni aynan 0 ga tushiradi
- Avtomatik feature tanlash

λ (lambda) – Regularizatsiya kuchi:
 λ katta → Kuchli regularizatsiya → Underfitting xavfi
 λ kichik → Zaif regularizatsiya → Overfitting xavfi
Optimal λ → Cross-validation bilan tanlanadi

Usul 2 — Ko'proq ma'lumot:

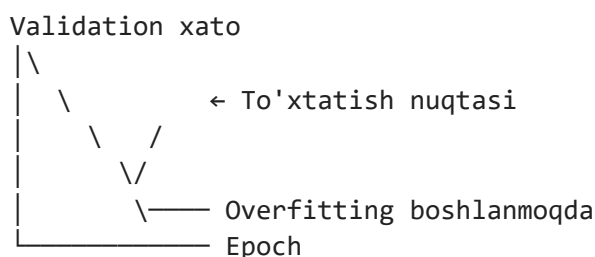
Overfitting ning asosiy sababi ko'pincha ma'lumot kamligi.
Ma'lumot ko'p → Model yodlash o'rniga o'rganishga majbur.

Ma'lumot augmentatsiyasi (rasmlarda):
→ Rasmni aylantirish, kichraytirish, yorug'lik o'zgartirish
→ Bir rasmdan ko'p variant yaratish

Usul 3 — Early Stopping (Neyron tarmoqlarda):

Neyron tarmoq o'rgatishda:
Birinchii epochlar: Train xato ↓, Validation xato ↓ (normal)
Ma'lum nuqtadan: Train xato ↓ davom etadi
Validation xato ↑ boshlanadi ← OVERFITTING!

Early Stopping: Validation xato oshishni boshlaganda to'xtatish



Usul 4 — Dropout (Neyron tarmoqlarda):

O'rgatishda tasodifiy neyronlarni o'chirish (masalan 50%)
→ Tarmoq har safar boshqacha → Yodlash qiyin
→ Natija: Umumlashtirilgan o'rganish

Test vaqtida: Barcha neyronlar ishlatiladi

Usul 5 — Feature Reduction:

Keraksiz o'zgaruvchilar → Overfitting xavfi oshadi
PCA (Principal Component Analysis) → O'lchamni kamaytirish
Feature Importance → Muhim o'zgaruvchilarni tanlash

8. Underfitting Oldini Olish Usullari

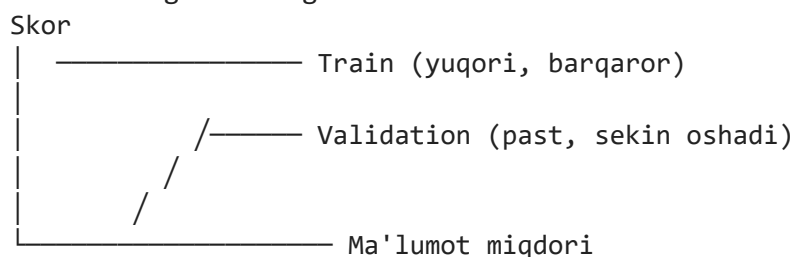
1. Murakkablikni oshirish:
 - Ko'proq parametrli model tanlash
 - Qaror daraxti uchun: max_depth oshirish
 - Neyron tarmoq uchun: qatlamlar/neyronlar qo'shish

2. Feature Engineering:
 - Yangi ma'lumot belgilari yaratish
 - X^2 yoki $X_1 \cdot X_2$ kabi o'zaro ta'sir atamalar
 - Logarifmik transformatsiya
 3. Regularizatsiyani kamaytirish:
 - λ ni kichraytirish
 - C ni oshirish (Logistik Regressiyada)
 4. Ko'proq o'rgatish:
 - Iteratsiya sonini oshirish
 - Learning rate ni kamaytirish
-

9. Aniqlash — Learning Curve

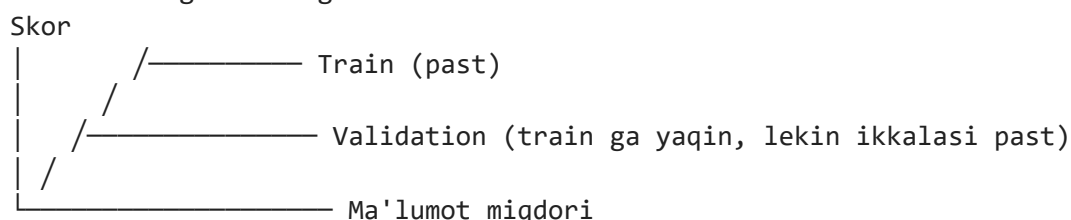
Learning Curve — train va validation skorlarini ma'lumot miqdoriga qarab ko'rsatish. Overfitting va Underfitting ni vizual aniqlash uchun eng kuchli vosita:

Overfitting Learning Curve:



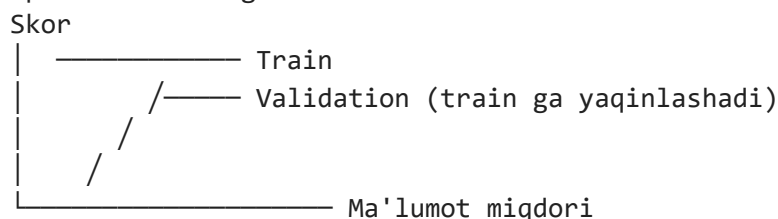
→ Katta bo'shliq: Ko'proq ma'lumot yoki regularizatsiya kerak

Underfitting Learning Curve:



→ Ikkalasi past: Murakkablikni oshirish kerak

Optimal Learning Curve:



→ Bo'shliq kichik, ikkalasi yuqori ✅

10. Xulosa

Train-Test Split:

- Model ko'rmagan ma'lumotda baholash
- Haqiqiy bashorat qobiliyatini o'lchash

→ 80/20 yoki 60/20/20 (train/val/test)

Overfitting:

- Train yaxshi, Test yomon → Katta farq
- Sabab: Murakkab model, kam ma'lumot
- Yechim: Regularizatsiya, ko'proq ma'lumot, Early Stopping, Dropout

Underfitting:

- Train ham, Test ham yomon → Ikkalasi past
- Sabab: Sodda model, kam feature
- Yechim: Murakkab model, Feature Engineering

Bias-Variance:

- Underfitting → Yuqori Bias, Past Variance
- Overfitting → Past Bias, Yuqori Variance
- Maqsad → Ikkalasi minimal bo'lgan nuqta

Aniqlash:

- Train va Test skori farqini ko'rish
- Learning Curve chizish
- Cross-Validation ishlatish

💡 **Eslab qol:** "Test to'plamiga bir marta tegish qoidasi" — Test to'plami faqat **eng oxirgi, yakuniy** baholash uchun. Agar test natijasiga qarab model o'zgartirilsa — test ham "train"ga aylanib qoladi va baholash ishonchsiz bo'ladi. Bu mashinali o'rganishdagi **eng muhim amaliy qoida!**